HAVEN

A SHARED-HOUSEHOLD APP FOR ROOMMATES.

TEAM MEMBERS:
CHRIS MARY BENSON (U56085268)
NANDANA SHASHI (U05556171)



What Haven is solving

Roommate Life is split across too many platforms!

- **Scattered Records**
  Who paid? Who owes? Where is the receipt?
- **Uneven workload**
  The same person ends up tracking and doing everything.
- **Forgotten follow-through**
  Payments and household tasks slip through the cracks.

THE HAVEN APPROACH

A shared home base for the whole household.

Everyone can see what's been logged, what's pending, and who is responsible.

DASHBOARD | EXPENSES | CHORES | GROCERIES

Current Status

IMPLEMENTED:

- USER LOGIN/ SIGNUP
- HOUSEHOLD SETUP (CREATE/JOIN)
- LEDGER TAB - TRACK, LOG & SETTLE EXPENSES
- DASHBOARD TAB - HOUSEHOLD AND PROFILE SETTINGS + LEDGER WIDGET

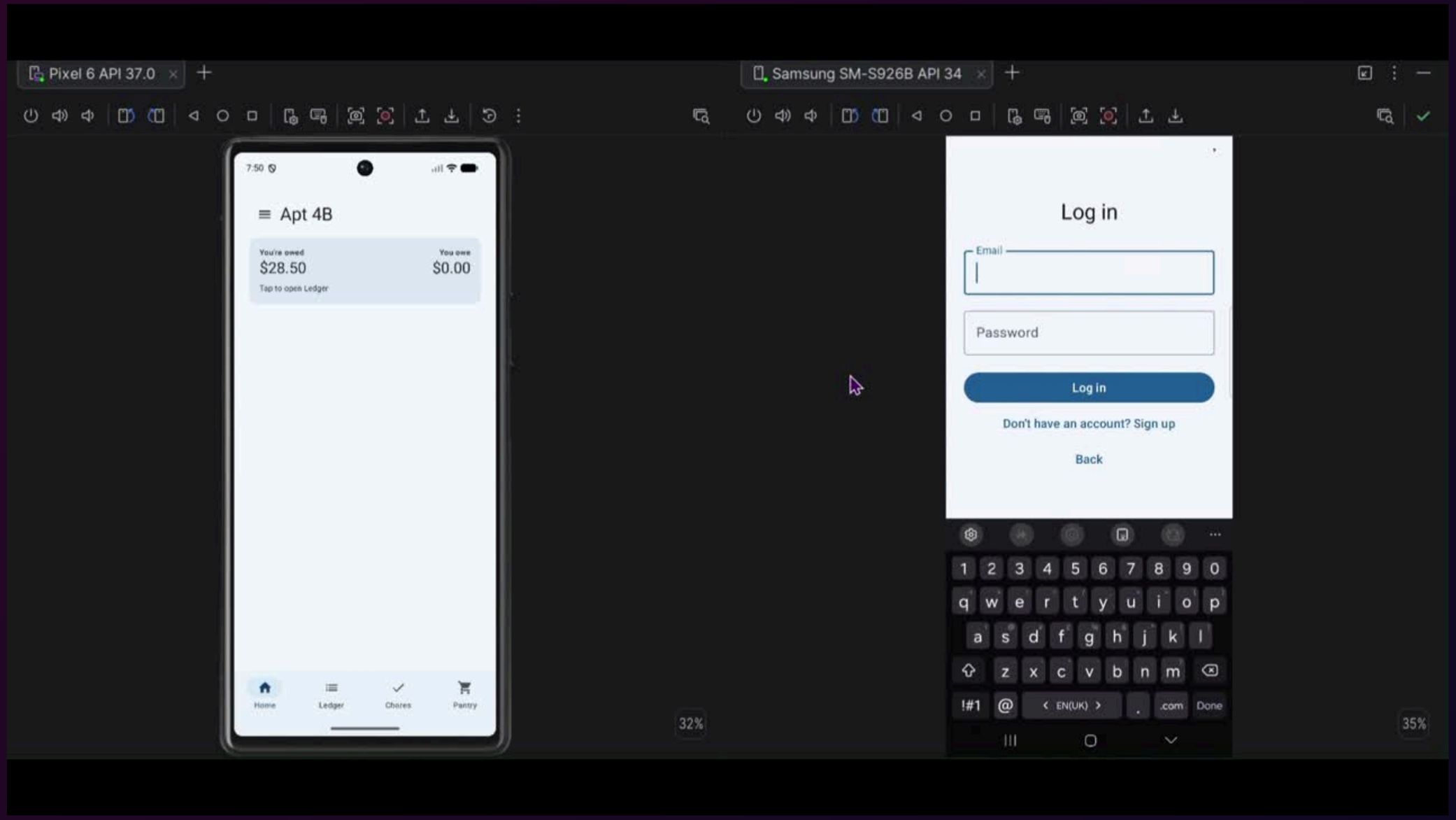
IN PROGRESS:

- CHORE ROTATION LOGIC + SCREENS, RENT TRACKER

TIMELINE

Weeks	Tasks
Oct 14 to Oct 20	~~Proposal Documentation, Repository Setup, Planning Tech Stack~~
Oct 21 to Oct 27	~~Data Model, Firebase & Room Setup, User Login/Signup & Joining household~~
Oct 28 to Nov 3	~~4 tab navigation, Expenses, Chore tab userflow implementation~~
Nov 4 to Nov 10	~~Dashboard view, Profile, Expenses~~, Chore tab userflow implementation, ~~prepare for Update 1~~
Nov 11 to Nov 17	Common Grocery List, Push Notifications, QR code scanning
Nov 18 to Nov 24	Push Notifications, Receipt Scanning, Prepare for Update 2
Nov 25 to Dec 1	UI/UX cleanup, Testing, Bug fixes, Finishing any other pending work
Dec 2 to Dec 8	Project Report, Testing, Demo Prep

LIVE DEMO


Pixel 6 API 37.0
Samsung SM-S926B API 34
7:50
Apt 4B
You're owed
$28.50
Tap to open Ledger
You owe
$0.00
Home
Ledger
Chores
Pantry
32%
Log in
Email
Password
Log in
Don't have an account? Sign up
Back
35%

ARCHITECTURE OVERVIEW

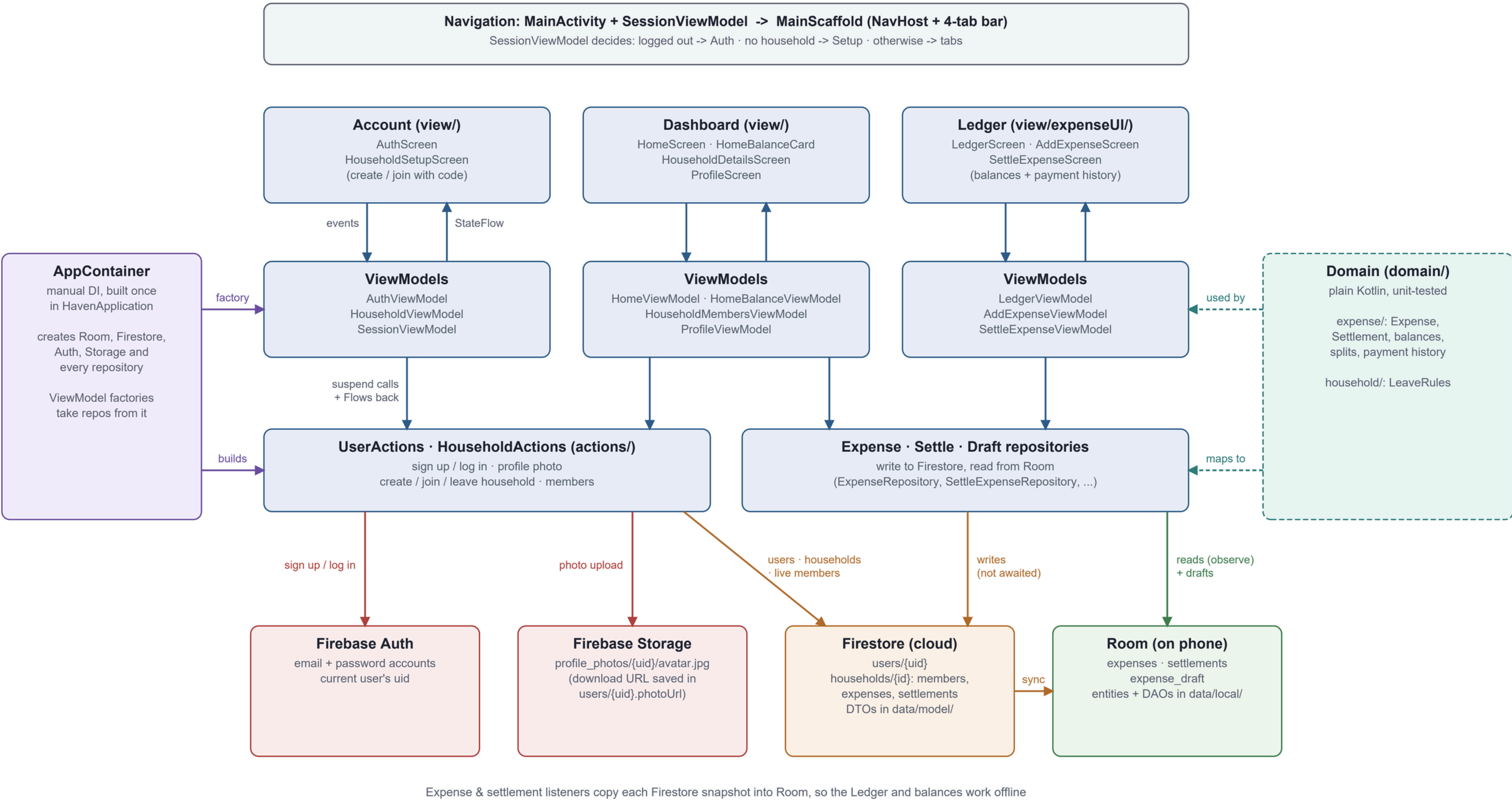
01 MVVM + Compose (Material 3)

02 Firestore writes, Room reads*

03 Manual dependency injection

DATA & PERSISTANCE

FEATURE	STORED IN	WHY
Accounts	Firebase Auth	email + password login, keeps the user signed in
Households, members, expenses, settlements, chores	Firestore	shared source of truth for all roommates
Local copy of expenses + settlements	Room	screens read from the phone's own database
Expense drafts	Room only	personal and unfinished, never shared
Profile photos	Firebase Storage	Firestore docs are limited to 1 MB, so only the URL goes there


Navigation: MainActivity + SessionViewModel -> MainScaffold (NavHost + 4-tab bar)
SessionViewModel decides: logged out -> Auth · no household -> Setup · otherwise -> tabs
Account (view/)
AuthScreen
HouseholdSetupScreen
(create / join with code)
Dashboard (view/)
HomeScreen · HomeBalanceCard
HouseholdDetailsScreen
ProfileScreen
Ledger (view/expenseUI/)
LedgerScreen · AddExpenseScreen
SettleExpenseScreen
(balances + payment history)
events
StateFlow
AppContainer
manual DI, built once
in HavenApplication
creates Room, Firestore,
Auth, Storage and
every repository
ViewModel factories
take repos from it
factory
ViewModels
AuthViewModel
HouseholdViewModel
SessionViewModel
ViewModels
HomeViewModel · HomeBalanceViewModel
HouseholdMembersViewModel
ProfileViewModel
ViewModels
LedgerViewModel
AddExpenseViewModel
SettleExpenseViewModel
used by
Domain (domain/)
plain Kotlin, unit-tested
expense/: Expense,
Settlement, balances,
splits, payment history
household/: LeaveRules
suspend calls
+ Flows back
builds
UserActions · HouseholdActions (actions/)
sign up / log in · profile photo
create / join / leave household · members
Expense · Settle · Draft repositories
write to Firestore, read from Room
(ExpenseRepository, SettleExpenseRepository, ...)
maps to
sign up / log in
photo upload
users · households
· live members
writes
(not awaited)
reads (observe)
+ drafts
Firebase Auth
email + password accounts
current user's uid
Firebase Storage
profile_photos/{uid}/avatar.jpg
(download URL saved in
users/{uid}.photoUrl)
Firestore (cloud)
users/{uid}
households/{id}: members,
expenses, settlements
DTOs in data/model/
sync
Room (on phone)
expenses · settlements
expense_draft
entities + DAOs in data/local/
Expense & settlement listeners copy each Firestore snapshot into Room, so the Ledger and balances work offline

Backlog

Feature Priority and Development status

P0 Core Features — Mostly Done

- Login & Sign up
- Household creation & joining
- Expense tracking & splitting
- Balances and settle-up
- Dashboard & Ledger
- Room + Firebase sync

Still to Do:
Chores feature · Shared grocery list

P1 Enhancements

- QR household joining
- Push notifications
- Receipt scanning

P2 Future Ideas

- Chore calendar
- Location-based grocery reminders

TASK	OWNER	STATUS
Chores feature	Chris + Nandana	Pending
Grocery list	Nandana	Pending
QR joining	Nandana	Planned
Push notifications	Chris	Planned
Receipt scanning	Both	Planned
Testing & UI polish	Both	Ongoing

Risks we are actively managing

The hard part is keeping shared data trustworthy.

		Risks	Mitigations
01	SYNC AND OFFLINE	Roommates may edit while offline or at the same time.	Firestore → Room sync with queued writes, last write wins, balances recalculated from all records.
02	MONEY ACCURACY	Uneven splits, rounding and settlements can create incorrect balances.	Store amounts in cents; validate shares; unit-test split and balance logic.
03	HOUSEHOLD LIFECYCLE	A member leaving must not break old expenses or balances.	Keep former-member records; lock affected historical expenses; test leave / settle-up paths.
04	PRIVACY & PERMISSIONS	Shared money data and photos could be seen by non-members; notifications need user permission (to be done)	Firestore security rules limit data to household members; the photo picker needs no permission; ask for notification permission during setup

Team Workflow

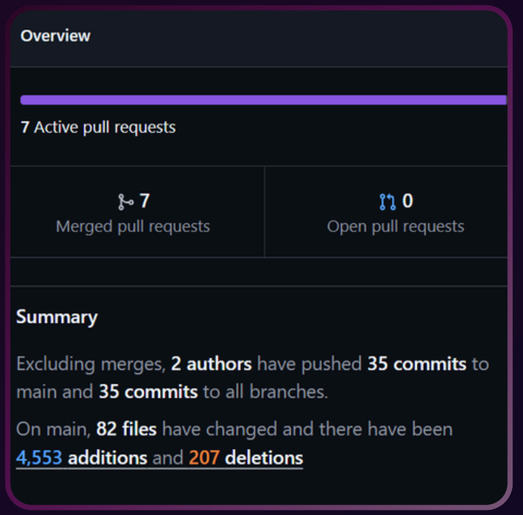
Chris

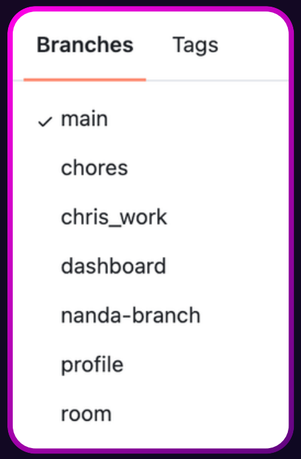
- Firestore setup & data model
- Expense logic & split logic
- Ledger screen & filters
- Add expense form & live members
- Settlement model & settle up flow
- Profile screen & photo upload

Nandana

- 4-tab navigation
- Dashboard & UI elements
- Login & sign-up screens
- Household details & invite code
- Welcome screen & profile pics
- Chores model & round robin logic

Aspect	Details	Status
Scrum Master	Rotates each week	In Progress
Meetings	1–2 check-ins a week to review progress, blockers and next tasks	On Track
Branching	One branch per feature (e.g. chris_work, rent-tracker), merged into main through pull requests	Implemented
Avoiding Conflicts	Pull main into feature branches often, so merges stay small	In Progress
Testing	Build and run unit tests before merging into main	On Track
Planning	Backlog prioritized against the Nov 10, Nov 24 and Dec 8 deadlines	In Progress





Thank You

Github Repository

https://github.com/nanda002/Haven.git